

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
ODD
Versione 1.5



Data: 21/12/2025

Progetto: MediBook	Versione: 1.5
Documento: ODD	Data: 21/12/2025

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Christian Casale
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
18/12/2025	1.1	Inizio stesura introduzione e package	Samuele Colucci
18/12/2025	1.2	Stesura e completamento package e class Interface	Samuele Colucci
18/12/2025	1.3	Stesura Componenti off-the-shelf e Design Patterns	Christian Casale
20/12/2025	1.4	Aggiunta diagrammi	Christian Casale
21/12/2025	1.5	Modifiche class interface	Christian Casale
11/01/2026	1.6	Cambio nome package	Lelio Crisci

Contents

1. Introduction	4
1.1. Object design trade-offs	4
1.1.1. Componenti off-the-shelf	5
1.1.2. Design Patterns	5
1.2. Interface documentation guidelines	6
1.3. Definitions, acronyms, and abbreviations	7
1.4. References	7
2. Packages	7
2.1. it.unisa.medibook.model	7
2.2. it.unisa.medibook.modelStorage	8
2.3. it.unisa.medibook.service	9
2.4. it.unisa.medibook.control	9
2.5. Viste (Presentation Layer - View)	10
3. Class interfaces	12
3.1. Package it.unisa.medibook.service	12
3.2. Package it.unisa.medibook.control	21
4. Glossary	22

Object Design Document

1. Introduction

1.1. Object design trade-offs

Durante la fase di Object Design del sistema MediBook, sono state prese decisioni architetturali che hanno richiesto un bilanciamento tra obiettivi di progettazione contrastanti. Di seguito sono riportati i principali compromessi (trade-offs) adottati:

- **Utilizzo Framework:** Si è scelto di utilizzare framework consolidati come **Spring Boot** e **Hibernate**.
 - *Compromesso:* Sebbene l'uso di framework introduca una maggiore complessità iniziale di configurazione e dipendenze esterne, garantisce una drastica riduzione dei tempi di sviluppo, maggiore sicurezza e aderenza agli standard industriali.
- **Sicurezza vs. Prestazioni (Security vs. Performance):** L'implementazione di filtri di sicurezza rigidi (Spring Security), l'hashing delle password con algoritmo BCrypt e l'obbligo di transazioni ACID sul database introducono un overhead computazionale (latenza aggiuntiva).
 - *Compromesso:* Si è deciso di sacrificare leggermente le prestazioni pure (tempo di risposta) per privilegiare l'**Integrità dei Dati** e la **Conformità GDPR**, ritenute prioritarie in un contesto medico-sanitario.
- **Astrazione vs. Controllo (ORM vs. SQL Puro):** Per la gestione dei dati è stato scelto l'approccio ORM (Object-Relational Mapping) tramite **JPA/Hibernate** anziché l'uso di query SQL native dirette.
 - *Compromesso:* Si accetta una minore ottimizzazione delle query complesse a favore di una maggiore velocità di sviluppo, portabilità del database e protezione automatica contro SQL Injection.

1.1.1. Componenti off-the-shelf

Per ottimizzare i tempi di sviluppo e garantire affidabilità, MediBook integra i seguenti componenti di terze parti:

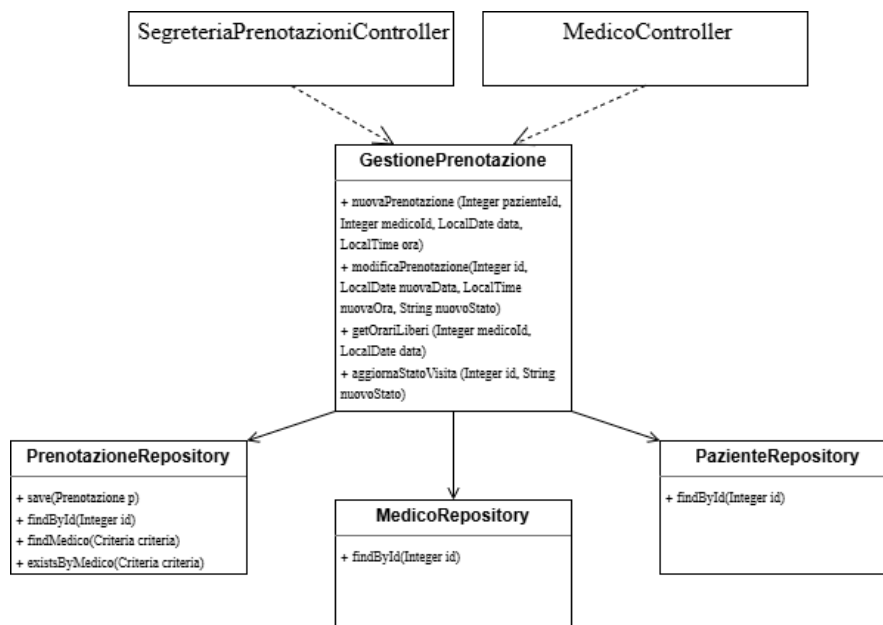
- **Spring Ecosystem (Boot, MVC, Security, Data JPA):** Gestisce l'intera infrastruttura di backend, fornendo configurazione automatica, pattern MVC, sicurezza (autenticazione/RBAC) e astrazione ORM per la persistenza dati.
- **MySQL Connector/J:** Driver JDBC essenziale per abilitare la connessione e le transazioni tra l'applicazione Java e il database MySQL.
- **Jakarta Mail:** Libreria standard utilizzata per l'invio automatizzato di notifiche email (es. conferme prenotazione, recupero credenziali).
- **Frontend Stack (Bootstrap 5 & JSTL):** JSTL gestisce la logica di visualizzazione dinamica all'interno delle pagine JSP.

1.1.2. Design Patterns

- **Gestione Prenotazione (Facade Pattern):**

La classe GestionePrenotazioni agisce come Facade per isolare la complessità della logica di prenotazione dai Controller. Il processo di creazione di una visita richiede l'interazione coordinata con diverse entità (Medico, Paziente) e la verifica di vincoli temporali complessi.

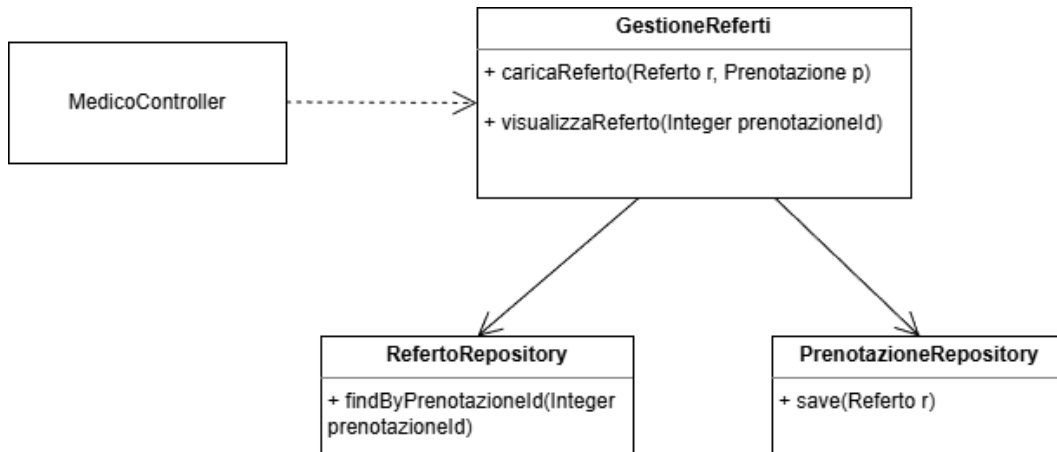
Diagramma:



- **Gestione Referti (Facade Pattern):**

La classe GestioneReferti implementa un Facade per garantire l'integrità referenziale e l'atomicità delle operazioni cliniche. Il metodo caricaReferto() incapsula una transazione critica che coinvolge più tabelle del database.

Diagramma:



1.2. Interface documentation guidelines

Per garantire la manutenibilità, la leggibilità e la coerenza tra i membri del team durante lo sviluppo del sistema con Java Spring Boot e JSP, sono state stabilite le seguenti convenzioni:

- **Naming Classi:** I nomi devono essere sostantivi singolari che rispecchiano l'entità del dominio (es. Prenotazione, Referto, Medico).
- **Naming Metodi:** I nomi devono contenere verbi che descrivono l'azione eseguita (es. effettuaPrenotazione(), caricaReferto(), findByData()).
- **Gestione Errori:** Gli errori di business (es. "Slot orario occupato") non devono essere gestiti tramite codici di ritorno numerici, ma attraverso il lancio di Eccezioni (Java Exceptions), catturate e gestite poi a livello di Controller o Service.
- **Accesso ai Dati (Repository Pattern):** L'accesso ai dati persistenti su MySQL attraverso Interfacce Repository che estendono JpaRepository (es. PrenotazioneRepository). Questo standardizza le operazioni CRUD e riduce il codice boilerplate.

1.3. Definitions, acronyms, and abbreviations

- ODD: Object Design Document.
- Repository: Pattern architetturale utilizzato per astrarre l'accesso ai dati persistenti
- DTO: Data Transfer Object (Oggetti per il passaggio dati tra i layer)
- API: Application Programming Interface
- MVC: Model-View-Controller
- JPA: Java Persistence API
- OCL: Object Constraint Language
- ORM: Object-Relational Mapping.

1.4. References

- Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd edition
- Slide del corso di Ingegneria del Software (Prof. De Lucia)
- Documento RAD
- Documento SDD
- Requisiti non funzionali: Sezione 3.3 del RAD

2. Packages

La decomposizione del sistema MediBook riflette un'architettura Web a tre livelli (Three-Tier Architecture) implementata secondo il pattern MVC (Model-View-Controller). Il codice sorgente Java è organizzato per separare nettamente le responsabilità di visualizzazione (Presentation Layer), logica di business (Business Layer) e persistenza dei dati (Persistence Layer).

2.1. `it.unisa.medibook.model`

Questo package contiene le classi Entity (POJO - Plain Old Java Objects) che rappresentano gli oggetti fondamentali del dominio e la struttura dei dati.

Classi Contenute:

- **Utente:** Questa classe rappresenta l'entità base per tutti gli attori del sistema. Definisce gli attributi comuni (come nome, cognome, email, password) e funge da superclasse.
- **Paziente:** Questa classe rappresenta l'attore che usufruisce dei servizi sanitari offerti dalla piattaforma. Estende l'entità **Utente** aggiungendo informazioni anagrafiche specifiche (es. codice fiscale, data di nascita, sesso) e mantiene le relazioni con lo storico delle proprie prenotazioni.
- **Medico:** Questa classe rappresenta il professionista sanitario operante all'interno della

struttura. Estende l'entità **Utente** includendo attributi professionali (es. specializzazione, numero albo) e gestisce la relazione con le visite assegnate e i referti emessi.

- **Prenotazione:** Questa classe rappresenta l'evento di una visita medica programmata. Costituisce l'associazione tra un **Paziente** e un **Medico** in uno specifico slot temporale (data e ora), tracciando lo stato del ciclo di vita della prestazione (es. *PRENOTATA*, *EFFETTUATA*, *CANCELLATA*).
- **Referto:** Questa classe rappresenta il documento clinico digitale redatto dal medico al termine di una prestazione. Contiene i dettagli della diagnosi e della terapia prescritta ed è associato in relazione 1-a-1 con una specifica **Prenotazione** conclusa.
- **Recensione:** Questa classe rappresenta il feedback qualitativo lasciato dal paziente dopo una visita. Memorizza la valutazione dell'esperienza e collega l'autore (**Paziente**) al destinatario della prestazione (**Medico**).
- **SegreteriaPrenotazioni:** Questa classe rappresenta l'attore amministrativo con privilegi specifici per la gestione dell'agenda. Estende **Utente** e identifica il personale autorizzato a supervisionare, modificare o cancellare le prenotazioni globali del sistema.
- **SegreteriaUtenti:** Questa classe rappresenta l'attore amministrativo incaricato della gestione delle anagrafiche. Estende **Utente** e identifica il personale responsabile della validazione dei profili medici e della gestione degli account dei pazienti.

2.2. **it.unisa.medibook.modelStorage**

Rappresenta il Data Access Layer (Persistenza). Contiene le interfacce Repository (basate su Spring Data JPA) che astraggono l'accesso al database.

Interfacce Contenute:

- **PrenotazioneRepository:** Gestisce la persistenza degli appuntamenti e le query per la verifica della disponibilità oraria.
- **MedicoRepository:** Gestisce il recupero dei profili medici e dei relativi turni di lavoro.
- **PazienteRepository:** Gestisce l'accesso ai dati anagrafici dei pazienti per l'associazione alle visite.
- **RefertoRepository:** Gestisce il salvataggio e il recupero dei referti medici associati alle visite concluse.
- **UtenteRepository:** Gestisce le credenziali di accesso e l'autenticazione per tutti i ruoli del sistema.
- **SegreteriaUtentiRepository:** Gestisce la ricerca e la manutenzione amministrativa degli

account (medici e pazienti) da parte dello staff.

- **SegreteriaPrenotazioniRepository**: Gestisce la supervisione globale e la modifica manuale di tutti gli appuntamenti presenti nel calendario del sistema.
- **RecensioneRepository**: Gestisce la memorizzazione e il recupero delle valutazioni lasciate dai pazienti.

2.3. **it.unisa.medibook.service**

Rappresenta il Service Layer (Business Logic). Contiene le classi di servizio che incapsulano la logica applicativa, le regole di validazione e il coordinamento delle transazioni.

Classi Contenute:

- **GestionePrenotazioni**: Il cuore del sistema. Valida la disponibilità degli slot temporali, impedisce sovrapposizioni, gestisce la creazione e la modifica delle visite (nuovaPrenotazione, modificaPrenotazione).
- **GestioneUtenza**: Gestisce la logica di autenticazione (login) e smistamento dei ruoli.
- **GestioneReferti**: Gestisce la logica di caricamento e associazione dei referti, verificando i vincoli di stato della visita (es. la visita deve essere "EFFETTUATA").
- **GestioneMedico**: Gestisce il profilo professionale e la configurazione dei turni lavorativi, dati essenziali per il calcolo delle disponibilità nel calendario.
- **EmailService**: Gestisce l'invio automatizzato di notifiche email.
- **PasswordService**: Gestisce la sicurezza delle credenziali tramite algoritmi di hashing, garantendo che le password vengano cifrate prima del salvataggio nel database.

2.4. **it.unisa.medibook.control**

Rappresenta il Controller Layer (Presentation). Contiene i Controller Spring MVC che gestiscono le richieste HTTP, invocano i Service e selezionano la vista (JSP) da renderizzare.

Classi Contenute:

- **AuthController**: Questa classe rappresenta il componente di gestione della sicurezza e dell'accesso. Gestisce le richieste HTTP relative all'autenticazione (login, logout) e alla registrazione di nuovi utenti, orchestrando la creazione e la distruzione delle sessioni utente.

- **PazienteController:** Questa classe rappresenta il controller dedicato alle funzionalità operative dell'attore Paziente. Gestisce la navigazione nella dashboard personale, la visualizzazione dello storico delle visite e le interazioni per la gestione del profilo e delle prenotazioni lato utente.
- **MedicoController:** Questa classe rappresenta il controller dedicato alle funzionalità operative dell'attore Medico. Gestisce la visualizzazione dell'agenda lavorativa (calendario), l'accesso ai dettagli dei pazienti e il flusso di compilazione e salvataggio dei referti medici.
- **SegreteriaPrenotazioniController:** Questa classe rappresenta il controller amministrativo focalizzato sulla gestione delle visite. Permette al personale di segreteria di supervisionare l'agenda globale, effettuando operazioni di modifica, cancellazione o conferma delle prenotazioni per conto di medici o pazienti.
- **SegreteriaUtentiController:** Questa classe rappresenta il controller amministrativo focalizzato sulla gestione delle anagrafiche. Gestisce le richieste per visualizzare la lista degli iscritti, approvare le registrazioni dei nuovi medici e gestire eventuali sospensioni o blocco degli account.
- **RicercaController:** Questa classe rappresenta il gestore della navigazione per la funzionalità di ricerca. Intercetta i filtri inseriti dall'utente e restituisce la vista (JSP) contenente l'elenco dei medici che soddisfano i criteri.
- **RicercaRestController:** Questa classe rappresenta l'interfaccia API REST per la ricerca asincrona. A differenza degli altri controller, non restituisce viste HTML ma dati in formato JSON, permettendo l'aggiornamento dinamico dei risultati senza ricaricare l'intera pagina web.

2.5. Viste (Presentation Layer - View)

Il layer di presentazione è costituito da pagine JSP (JavaServer Pages) locate in /webapp/WEB-INF/view/.

File Principali:

- **cambio_password_obbligatorio.jsp:** Modulo di cambio password dopo primo accesso di utente registrato da segreteria.
- **dashboard-agenda.jsp:** Dashboard agenda del della segreteriaPrenotazioni
- **dashboard-gestione-anagrafiche.jsp:** Dashboard modifica dati utente da parte della segreteria utenti.
- **dettagli_medico.jsp:** Pagina di dettaglio accessibile ai pazienti.
- **form_paziente.jsp:** Modulo riutilizzabile per l'inserimento o la modifica dei dati

anagrafici di un paziente utilizzato sia dalla segreteria utenti.

- **home.jsp:** Home Page dell'applicazione web.
- **login.jsp:** Modulo di accesso unificato.
- **medico.jsp:** Interfaccia di gestione visite per il medico.
- **medico_scrivi_referto.jsp:** Form di scrittura e salvataggio referto da parte del medico.
- **medicoCalendario.jsp:** Modulo per visualizzazione calendario del medico.
- **paziente.jsp:** Dashboard interattiva per il paziente.
- **paziente_profilo.jsp:** Area riservata a paziente dedicata alla gestione dati da parte del paziente.
- **paziente_visualizza_referto.jsp:** Pagina di lettura referto da parte del paziente.
- **registrazione.jsp:** Modulo di registrazione.
- **risultatiRicerca.jsp:** Pagina visualizzazione lista medici dopo la ricerca.
- **segreteria_scrivi_referto.jsp:** Form di scrittura e salvataggio referto da parte della segreteria utenti.
- **segreteria_utenti.jsp:** Dashboard amministrativa per la gestione degli utenti da parte della segreteria utenti.
- **visualizzaRefertoMedico.jsp:** Pagina di lettura referto da parte del medico.

3. Class interfaces

3.1. Package `it.unisa.medibook.service`

Questo package contiene i Controller. Qui l'enfasi è sulle Pre-condizioni (cosa deve essere vero prima di chiamare il metodo) e Post-condizioni (cosa è garantito dopo).

GestionePrenotazioni	
modificaPrenotazione	Context: <code>GestionePrenotazioni::modificaPrenotazione(id, nuovaData, nuovaOra, nuovoStato): Prenotazione</code> Pre: ValidazioneInput: -- 1. La prenotazione deve esistere <code>self.prenotazioneRepository.existsById(id)</code> and -- 2. La nuova data non può essere nel passato <code>nuovaData >= LocalDate.now()</code> and -- 3 Lo slot non deve essere occupato da ALTRE prenotazioni <code>not self.prenotazioneRepository.existsByMedicoIdAndDataAndOraAndIdNot(</code> <code>self.prenotazioneRepository.findById(id).medico.id,</code> <code>nuovaData,</code> <code>nuovaOra,</code> <code>id</code> <code>)</code> Post: RisultatoAtteso: -- Il risultato deve essere l'oggetto aggiornato e salvato <code>result.id = id</code> and <code>result.data = nuovaData</code> and <code>result.ora = nuovaOra</code> and <code>result.stato = nuovoStato</code> and <code>self.prenotazioneRepository.findById(id) = result</code>
aggiornaStatoVisita	Context: <code>GestionePrenotazioni::aggiornaStatoVisita(id: Integer, nuovoStato: String): Prenotazione</code> Pre: ValidazioneInput: -- La prenotazione deve esistere per poter cambiare stato <code>self.prenotazioneRepository.existsById(id)</code> Post: StatoAggiornato: -- Il campo stato deve essere quello nuovo passato come parametro

	<pre> result.stato = nuovoStato and -- L'ID deve rimanere invariato result.id = id and -- Data e Ora non devono essere cambiate rispetto a prima (@pre) result.data = self.prenotazioneRepository.findById(id)@pre.data and result.ora = self.prenotazioneRepository.findById(id)@pre.ora and -- Verifica di persistenza self.prenotazioneRepository.findById(id) = result </pre>
visualizzaVisiteMedico	Context: GestionePrenotazioni::visualizzaVisiteMedico(medicoId: Integer): Sequence(Prenotazione) Pre: MedicoEsistente: -- Il medico di cui cerchiamo le visite deve esistere self.medicoRepository.existsById(medicoId) Post: FiltroCorretto: -- Ogni prenotazione nella lista deve avere l'ID medico uguale a quello richiesto result->forAll(p p.medico.id = medicoId)
visualizzaVisitePaziente	Context: GestionePrenotazioni::visualizzaVisitePaziente(pazienteId: Integer): Sequence(Prenotazione) Pre: IdNonNullo: -- L'ID del paziente deve essere fornito pazienteId <> null Post: RisultatoCoerente: -- 1. Tutti gli elementi della lista restituita devono appartenere a quel paziente result->forAll(p p.paziente.id = pazienteId) and -- 2. Il risultato deve corrispondere esattamente ai dati nel DB result = self.prenotazioneRepository.findByPazienteId(pazienteId)
nuovaPrenotazione	Context: GestionePrenotazioni::nuovaPrenotazione(pazienteId: Integer, medicoId: Integer, data: LocalDate, ora: LocalTime): Prenotazione Pre: ValidazioneInserimento: -- 1. La data non può essere nel passato data >= LocalDate.now() and -- 2. Il medico e il paziente devono esistere nel DB self.medicoRepository.existsById(medicoId) and self.pazienteRepository.existsById(pazienteId) and -- 3. Lo slot orario deve essere libero per quel medico

	not self.prenotazioneRepository.existsByMedicoIdAndDataAndOra(medicoId, data, ora) Post: NuovaPrenotazioneCreata: -- 1. Verifica che i dati dell'oggetto restituito siano corretti result.data = data and result.ora = ora and result.stato = 'PRENOTATA' and -- 2. Verifica le associazioni (Foreign Keys logiche) result.paziente.id = pazienteId and result.medico.id = medicoId and -- 3. Verifica che il numero totale di prenotazioni nel DB sia aumentato di 1 self.prenotazioneRepository.count() = self.prenotazioneRepository.count()@pre + 1 and -- 4. Verifica di persistenza self.prenotazioneRepository.findById(result.id) = result
getGiorniLavorativi	Context GestionePrenotazioni::getGiorniLavorativi(medicoId: Integer): Sequence(Integer) Pre MedicoEsistente: -- Il medico deve esistere per poter leggere i suoi turni self.medicoRepository.existsById(medicoId) Post GiorniSettimanaValidi: -- Tutti i numeri restituiti devono essere giorni validi (1=Lun ... 7=Dom) result->forall(giorno giorno >= 1 and giorno <= 7) and -- Ogni giorno restituito deve essere presente nella stringa 'turni' del medico let turniMedico: String = self.medicoRepository.findById(medicoId).turni in result->forall(g turniMedico.indexOf(g.toString()) > 0)
visualizzaVisitePerCalendario	Context GestionePrenotazioni::visualizzaVisitePerCalendario(medicoId: Integer): Sequence(Prenotazione) Pre MedicoEsistente: self.medicoRepository.existsById(medicoId) Post FiltroStatoAttivo: -- Ogni prenotazione 'p' restituita deve appartenere al medico E con stato -- 'PRENOTATA' result->forall(p p.medico.id = medicoId and p.stato = 'PRENOTATA')

getOrariLiberi	Context GestionePrenotazioni::getOrariLiberi(medicId: Integer, data: LocalDate): Sequence(LocalTime) Pre MedicoEsistente: -- Il medico deve esistere per calcolarne i turni self.medicoRepository.existsById(medicId) Post SlotNonOccupati: -- Nessuno degli orari restituiti (result) deve essere presente nel database delle prenotazioni result->forAll(t not self.prenotazioneRepository.existsByMedicoIdAndDataAndOra(medicId, data, t)) and -- Poiché OCL non può fare il parsing della stringa "turni" (es. "1:09-13"), -- astraiano dicendo che ogni orario restituito 't' deve essere compreso -- nella definizione dei turni del medico per quella data. let medico: Medico = self.medicoRepository.findById(medicId) in result->forAll(t -- (Specifica astratta) L'orario t rientra nella stringa dei turni configurata medico.turniInclude(data.getDayOfWeek(), t))
----------------	---

GestioneUtenza	
registraPaziente	Context: GestioneUtenza::registraPaziente(p: Paziente) Pre: ValidazioneNuovoUtente: -- L'oggetto paziente non deve essere nullo p <> null and -- L'email non deve essere già associata a nessun altro utente (medico o paziente) -- Nota: Usiamo 'not' perché se esiste, il metodo lancerebbe eccezione not self.utenteRepository.existsByEmail(p.email) Post: RegistrazioneEffettuata: -- 1. Il ruolo viene forzato a PAZIENTE (sovrascrivendo qualsiasi altro valore) p.ruolo = 'PAZIENTE' and -- 2. Il paziente è stato salvato nel database self.pazienteRepository.existsById(p.id) and -- 3. Il numero totale di pazienti nel sistema è aumentato di 1

	self.pazienteRepository.count() = self.pazienteRepository.count()@pre+1
--	---

GestioneReferti	
visualizzaReferto	Context: GestioneReferti::visualizzaReferto(prenotazioneId: Integer): Referto Pre: IdValido: -- L'ID passato non deve essere nullo prenotazioneId $\neq$ null and -- La prenotazione a cui cerchiamo il referto deve esistere self.prenotazioneRepository.existsById(prenotazioneId) Post: CorrispondenzaDati: -- SE il risultato non è nullo, ALLORA deve appartenere alla -- prenotazione richiesta result $\neq$ null implies result.prenotazione.id = prenotazioneId and -- Il risultato deve corrispondere esattamente a quello memorizzato -- nel database result = self.refertoRepository.findByPrenotazioneId(prenotazioneId)
caricaReferto	Context GestioneReferti::caricaReferto(r: Referto, p: Prenotazione): Referto Pre ValidazioneBusiness: -- 1. I parametri in input devono essere validi (non nulli) r $\neq$ null and p $\neq$ null and -- 2. Si può refertare solo una visita che è stata fisicamente eseguita. -- Non è possibile caricare referti su visite future ('PRENOTATA') o cancellate. p.stato = 'EFFETTUATA' Post TransazioneCompletata: -- 1. Side Effect: Lo stato della prenotazione deve avanzare -- automaticamente a 'CONCLUSA' p.stato = 'CONCLUSA' and -- 2. Il referto deve essere collegato correttamente alla prenotazione -- passata in input r.prenotazione = p and -- 3. Verifica di persistenza: il referto è stato salvato nel database self.refertoRepository.existsById(r.id) and result = r

GestioneMedico	
dammiTuttiIMedici	Context: GestioneMedico::dammiTuttiIMedici(): Sequence(Medico) Pre: true -- Nessun vincolo specifico per l'invocazione Post: ListaCompleta: -- 1. Il risultato deve corrispondere esattamente al contenuto del repository result = self.medicoRepository.findAll() and -- 2. La dimensione della lista restituita deve coincidere con il numero totale di record nel DB result->size() = self.medicoRepository.count()
getMedicoById	Context: GestioneMedico::getMedicoById(id: Integer): Medico Pre: IdFornito: -- L'ID passato come parametro non deve essere nullo id <> null Post: RisultatoCoerente: -- Se il medico esiste nel database, il risultato non deve essere nullo self.medicoRepository.existsById(id.toLong()) implies result <> null and -- Se il risultato non è nullo, l'ID dell'oggetto restituito deve -- corrispondere all'ID richiesto result <> null implies result.id = id.toLong() and -- Il risultato deve corrispondere esattamente a quanto restituito -- dalla repository result = self.medicoRepository.findById(id.toLong())

EmailService

inviaEmailModifica	Context EmailService::inviaEmailModifica(destinatario: String, oggetto: String, nomePaziente: String, nomeMedico: String, nuovaData: String, nuovaOra: String) Pre InputValidi: -- 1. L'indirizzo email del destinatario è obbligatorio e non vuoto destinatario <> null and destinatario.size() > 0 and -- 2. I dati fondamentali della modifica non devono essere nulli nuovaData <> null and nuovaOra <> null Post EsecuzioneSenzaErrori: -- Poiché il metodo gestisce internamente le eccezioni (try-catch), -- la post-condizione garantisce che il flusso di controllo ritorni al -- chiamante -- senza interrompere l'esecuzione del programma. true -- (Opzionale: Specifica astratta dell'invocazione) -- Implica che la libreria mailSender sia stata chiamata -- self.mailSender.sentMessages->includes(message)
inviaEmailConferma	Context: EmailService::inviaEmailConferma(destinatario: String, nomePaziente: String, nomeMedico: String, data: String, ora: String) Pre: DatiEmailValidi: -- Il destinatario deve esistere e non essere vuoto destinatario <> null and destinatario.size() > 0 and -- I parametri per compilare il template HTML non devono essere nulli nomePaziente <> null and nomeMedico <> null and data <> null and ora <> null Post: RichiestaInviata: -- Nota: OCL non può verificare se l'SMTP ha consegnato realmente l'email. -- Si assume che il comando 'mailSender.send()' sia stato invocato correttamente. true
inviaEmailBenvenuto	Context EmailService::inviaEmailBenvenuto(destinatario: String, nomeUtente: String) Pre DatiContattoValidi: -- L'indirizzo email è obbligatorio per l'invio destinatario <> null and destinatario.size() > 0

	and -- Il nome è necessario per personalizzare il template (es. "Ciao --Mario") nomeUtente <> null Post EsecuzioneCompletata: -- Il metodo garantisce la terminazione del flusso senza interrompere -- il processo di registrazione chiamante, gestendo internamente -- eventuali errori SMTP. true
costruisciHtmlModifica	Context: EmailService::costruisciHtmlModifica(nome: String, medico: String, data: String, ora: String): String Pre: ParametriValidi: -- I parametri non devono essere nulli nome <> null and medico <> null and data <> null and ora <> null Post: ContenutoDinamicoPresente: -- La stringa risultante deve includere tutti i dati variabili passati result.indexOf(nome) >= 0 and result.indexOf(medico) >= 0 and result.indexOf(data) >= 0 and result.indexOf(ora) >= 0 and -- Il risultato deve essere un documento HTML completo result.indexOf('<html>') >= 0 and result.indexOf('</html>') >= 0
inviaEmailModificaMedico	Context: EmailService::inviaEmailModificaMedico(destinatarioMedico: String, nomePaziente: String, nuovaData: String, nuovaOra: String) Pre: ParametriObbligatori: -- L'indirizzo email del medico deve essere presente destinatarioMedico <> null and destinatarioMedico.size() > 0 and -- I dettagli dell'appuntamento per il template HTML non devono essere nulli nomePaziente <> null and nuovaData <> null and nuovaOra <> null Post: NotificaGestita: -- Il metodo completa l'esecuzione. -- L'eventuale fallimento dell'invio (SMTP) viene catturato e loggato, -- quindi non influisce sullo stato degli oggetti del dominio. true

PasswordService

hash	Context PasswordService::hash(passwordInChiaro: String): String Pre InputValido: -- La password da cifrare non deve essere nulla o vuota passwordInChiaro <> null and passwordInChiaro.size() > 0 Post SicurezzaGarantita: -- 1. Il metodo deve restituire una stringa valida result <> null and -- 2. L'hash generato deve essere DIVERSO dalla password originale. -- Questo garantisce che non stiamo salvando testo in chiaro nel -- DB. result <> passwordInChiaro
check	Context: PasswordService::check(passwordInserita: String, passwordNelDatabase: String): Boolean Pre: InputValidi: -- Entrambe le stringhe devono essere fornite per il confronto passwordInserita <> null and passwordNelDatabase <> null Post: RisultatoConvalida: -- Il risultato è vero se e solo se la password fornita corrisponde -- all'hash nel DB result = (self.passwordEncoder.matches(passwordInserita, passwordNelDatabase)) and passwordInserita = passwordInserita@pre and passwordNelDatabase = passwordNelDatabase@pre

3.2.

Package it.unisa.medibook.control

PasswordService	
performLogin	<pre> context AuthController::performLogin(email: String, password: String, redirect: String, cerca: String, session: HttpSession): String pre InputValidi: -- L'email e la password inserite non devono essere nulle o vuote email <> null and email.size() > 0 and password <> null and password.size() > 0 post AutenticazioneFallita: -- Se le credenziali non sono corrette, il sistema ritorna alla pagina di login con un errore (self.utenteRepository.findByEmail(email) = null or not self.passwordService.check(password, self.utenteRepository.findByEmail(email).password)) implies result = 'login' post AutenticazioneRiuscita: -- Se le credenziali sono corrette, l'utente viene messo in sessione let utenteLoggato : Utente = self.utenteRepository.findByEmail(email) in (utenteLoggato <> null and self.passwordService.check(password, utenteLoggato.password)) implies (session.getAttribute('utente') = utenteLoggato and -- Gestione dei Redirect specifici if password = 'Medibook123' then result = 'redirect:/cambio-password-obbligatorio' else if utenteLoggato.ocllsKindOf(SegreteriaUtenti) then result = 'redirect:/segreteria-utenti/dashboard' else if utenteLoggato.ruolo = 'MEDICO' then result = 'redirect:/medico' else -- Caso Paziente (gestisce anche il parametro cerca) result.startsWith('redirect:/paziente') endif) </pre>

4. Glossary

Questa sezione fornisce le definizioni dei termini tecnici, degli acronimi e delle tecnologie menzionate all'interno del presente documento di Object Design.

- **BCrypt**: Algoritmo di hashing forte utilizzato all'interno di PasswordService per la cifratura delle password prima della persistenza nel database.
- **CSS (Cascading Style Sheets)**: Linguaggio utilizzato per definire la formattazione e lo stile grafico dei componenti dell'interfaccia utente.
- **DBMS (Database Management System)**: Software progettato per la gestione dei dati; MediBook utilizza **MySQL 8.0**.
- **Invariante (Invariant)**: Vincolo OCL che deve risultare sempre valido per tutte le istanze di una determinata classe.
- **Java**: Linguaggio di programmazione orientato agli oggetti utilizzato per l'intero backend e la logica di business.
- **JavaBeans (Entity)**: Classi che incapsulano i dati del dominio (es. Prenotazione, Paziente), mappate sul database tramite annotazioni JPA.
- **JSON (JavaScript Object Notation)**: Formato per lo scambio di dati utilizzato dalle API REST per aggiornamenti asincroni della UI.
- **JSP (JavaServer Pages)**: Tecnologia utilizzata dai Controller per generare dinamicamente le pagine HTML lato server.
- **MVC (Model-View-Controller)**: Pattern architetturale che separa la gestione dei dati (Model), l'interfaccia utente (View) e la logica di controllo (Controller).
- **OCL (Object Constraint Language)**: Linguaggio formale che permette di specificare vincoli su singoli elementi del modello o gruppi di essi. È un linguaggio non procedurale e non denota il flusso di controllo.
- **Post-condizione (Postcondition)**: Vincolo OCL che definisce lo stato del sistema o il valore di ritorno (result) dopo l'esecuzione di un metodo.
- **Pre-condizione (Precondition)**: Vincolo OCL che deve essere soddisfatto affinché un metodo possa essere eseguito correttamente.
- **Repository (Spring Data JPA)**: Interfacce che gestiscono automaticamente l'accesso ai dati e le query al database.

- **Service Layer:** Livello architetturale che incapsula la Business Logic (es. GestionePrenotazioni), coordinando le transazioni e applicando le regole definite nei contratti OCL.
- **Spring Boot:** Framework utilizzato per semplificare la configurazione e il deployment dell'applicazione Java.
- **SQL (Structured Query Language):** Linguaggio standard per l'interrogazione dei database relazionali.